

# Iterating functions

DAX FUNCTIONS IN POWER BI



**Maarten Van den Broeck**

Content Developer at DataCamp

# Iterating functions

- Iterate over each row of a given table to perform an expression

`SUMX(<table>, <expression>)`   `AVERAGEX(<table>, <expression>)`

- X stands for eXpression
- Allow for advanced calculations specified at each row

# Iterating functions: SUMX()

## Calculated column example

```
Cost = Fact_Orders[Sales] - Fact_Orders[Profit]
```

```
Total Costs = SUM(Fact_Orders[Cost])
```

| Sales       | Profit  | Cost    |
|-------------|---------|---------|
| \$77.88     | \$3.89  | \$73.99 |
| \$22.72     | \$10.22 | \$12.50 |
| ...         | ...     | ...     |
| Total Costs |         |         |
| \$2,569     |         |         |

# Iterating functions: SUMX()

## Calculated column example

```
Cost = Fact_Orders[Sales] - Fact_Orders[Profit]
```

```
Total Costs = SUM(Fact_Orders[Cost])
```

| Sales       | Profit  | Cost    |
|-------------|---------|---------|
| \$77.88     | \$3.89  | \$73.99 |
| \$22.72     | \$10.22 | \$12.50 |
| ...         | ...     | ...     |
| Total Costs |         |         |
| \$2,569     |         |         |

## Iterating function example

```
Total Costs SUMX =  
SUMX(Fact_Orders,  
      Fact_Orders[Sales] - Fact_Orders[Profit])
```

**Total Costs SUMX**

\$2,569

# Filtering iterating functions

- Use filter functions, such as FILTER(), to return a filtered table

```
SUMX(  
  FILTER(  
    <table>,  
    <filter>),  
  <expression>)
```

```
Total Costs East SUMX =  
SUMX(  
  FILTER(  
    Fact_Orders,  
    Fact_Orders[Region] = "East"),  
  Fact_Orders[Sales] - Fact_Orders[Profit])
```

# Filtering iterating functions

- Use filter functions, such as FILTER(), to return a filtered table

```
SUMX(  
  FILTER(  
    <table>,  
    <filter>),  
  <expression>)
```

```
Total Costs East SUMX =  
SUMX(  
  FILTER(  
    Fact_Orders,  
    Fact_Orders[Region] = "East"),  
  Fact_Orders[Sales] - Fact_Orders[Profit])
```

| Region  | Total Costs    | Total Costs East SUMX |
|---------|----------------|-----------------------|
| Central | \$501,239.89   |                       |
| East    | \$678,781.24   | \$678,781.24          |
| South   | \$391,721.91   |                       |
| West    | \$725,457.82   |                       |
| TOTAL   | \$2,297,200.86 | \$678,781.24          |

# Iterating functions: RANKX()

```
RANKX(  
    <table>,  
    <expression>)
```

- Rank regions by total costs

```
Total Costs RANKX =  
RANKX(  
    ALL(Dim_Sales[Region]),  
    [Total Costs])
```

- Use `ALL()` to evaluate all rows from the dimension table



# Iterating functions: RANKX()

```
RANKX(  
    <table>,  
    <expression>)
```

- Rank regions by total costs

```
Total Costs RANKX =  
RANKX(  
    ALL(Dim_Sales[Region]),  
    [Total Costs])
```

- Use `ALL()` to evaluate all rows from the dimension table

| Region  | Total Costs  | Total Costs RANKX |
|---------|--------------|-------------------|
| Central | \$725,457.82 | 1                 |
| East    | \$678,781.24 | 2                 |
| South   | \$501,239.89 | 3                 |
| West    | \$391,721.91 | 4                 |



# Operators in DAX

## COMPARISON OPERATORS

| Operator | Meaning                  |
|----------|--------------------------|
| =        | Equal to                 |
| ==       | Strict equal to          |
| >        | Greater than             |
| <        | Smaller than             |
| >=       | Greater than or equal to |
| <=       | Smaller than or equal to |
| <>       | Not equal to             |

# Operators in DAX

## COMPARISON OPERATORS

| Operator | Meaning                  |
|----------|--------------------------|
| =        | Equal to                 |
| ==       | Strict equal to          |
| >        | Greater than             |
| <        | Smaller than             |
| >=       | Greater than or equal to |
| <=       | Smaller than or equal to |
| <>       | Not equal to             |

## TEXT OPERATOR

| Operator | Meaning                  | Example                 |
|----------|--------------------------|-------------------------|
| &        | Concatenates text values | [City]&", "&<br>[State] |

# Operators in DAX

## COMPARISON OPERATORS

| Operator | Meaning                  |
|----------|--------------------------|
| =        | Equal to                 |
| ==       | Strict equal to          |
| >        | Greater than             |
| <        | Smaller than             |
| >=       | Greater than or equal to |
| <=       | Smaller than or equal to |
| <>       | Not equal to             |

## TEXT OPERATOR

| Operator | Meaning                  | Example              |
|----------|--------------------------|----------------------|
| &        | Concatenates text values | [City]&"", "&[State] |

## LOGICAL OPERATORS

| Operator | Meaning                   | Example                                   |
|----------|---------------------------|-------------------------------------------|
| &&       | AND condition             | ([City] = "Bru") && ([Return] = "Yes"))   |
|          | OR condition              | ([City] = "Bru")    ([Return] = "Yes"))   |
| IN { }   | OR condition for each row | Product[Color] IN {"Red", "Blue", "Gold"} |

**Lesson[Knowledge]  
IN {"Poor", "Great",  
"Awesome!"}**

DAX FUNCTIONS IN POWER BI

# Iterating functions in Power BI

DAX FUNCTIONS IN POWER BI



**Maarten Van den Broeck**

Content Developer at DataCamp

# Let's practice!

DAX FUNCTIONS IN POWER BI

# Congratulations!

DAX FUNCTIONS IN POWER BI



**Carl Rosseel**  
Curriculum Manager



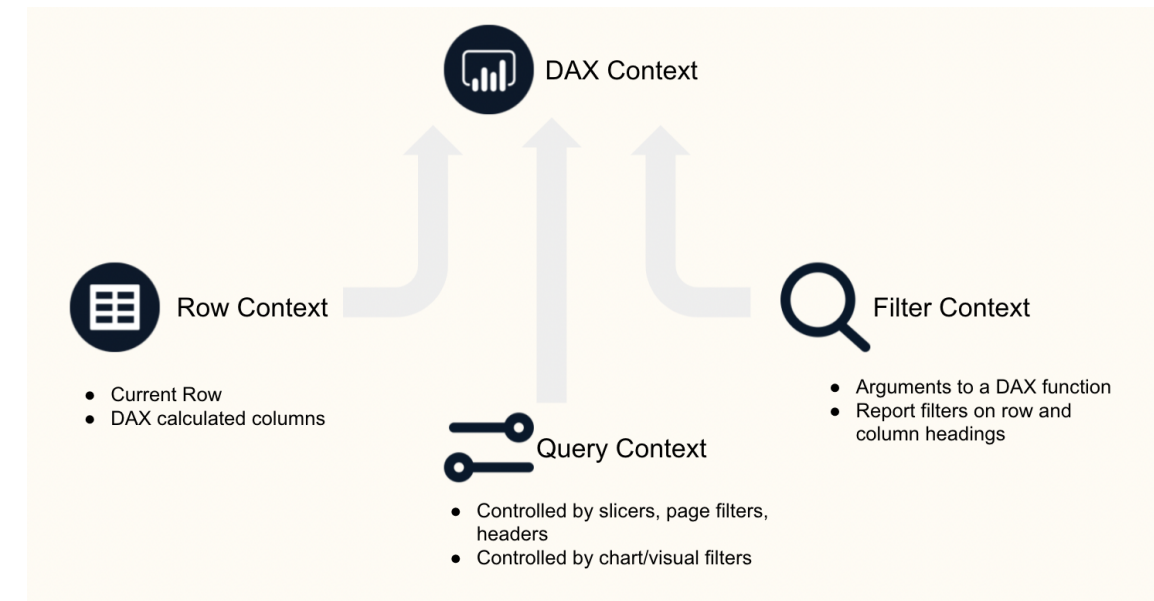
# DAX stands for Data Analysis eXpressions

DAX formulas are used in:

- Measures
- Calculated columns
- Calculated tables

Context in DAX Formulas:

- Row context
- Query context
- Filter context





# DAX Toolbox

## General:

- Implicit vs explicit measures
- Quick measures
- Variables: `VAR`

## Data Modeling:

- `CALENDAR()`
- `CALCULATE()`
- `RELATED()`
- `FILTER()`
- `CROSSFILTER()`

# Enjoy iterating!

DAX FUNCTIONS IN POWER BI